CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)
CSPIT — Department of Computer Engineering · ITUE203 Web Development Frameworks

Student Name	Dhruv Bhoi	**Student ID**	**D26CE154**
Academic Year	2026–27 (ODD)	**Semester**	3rd Semester, B.Tech CE

Practical 10: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Practical Aim & Tools

Aim: Implement secure authentication (login/logout) using PHP sessions, `password_verify()`, session regeneration against fixation, role-based access control (RBAC), 15-minute inactivity timeout, and secure remember-me cookies.

Tools & Technologies: PHP Sessions, Cryptographic Tokens, HTTPOnly/Secure Cookies, RBAC Architecture

1. Session Management & Defense-in-Depth Authentication

StudentHub (`login.php`, `logout.php`, `includes/functions.php`) enforces enterprise security standards:

• Secure Authentication: Uses `password_verify($password, $hash)` resisting timing attacks.
• Session Fixation Defense: Calls `session_regenerate_id(true)` upon successful authentication to destroy the pre-login session ID.
• Inactivity Timeout: Automatically tracks `$_SESSION['last_activity']`. Sessions exceeding 15 minutes (`SESSION_TIMEOUT = 900`) are destroyed with a timeout redirect.
• Split-Token Remember-Me: Stores a public 12-byte hex selector and a private 32-byte token hash in `remember_tokens`. Cookies are marked `HttpOnly`, `SameSite=Lax`, and `Expires = 14 days`.
• Role-Based Access Control (RBAC): `require_role('admin')` blocks students from administrative scripts (`students.php`, `manage_events.php`, `admin/`).

2. Implementation Code: Secure Login & Remember-Me

Listing: login.php — Authentication, Session Regeneration, and Split-Token Generation

```
// login.php - Authentication & Token Issue
$st = $pdo->prepare('SELECT * FROM users WHERE email = ? AND status = "active"');
$st->execute([$email]);
$u = $st->fetch();

if ($u && password_verify($pass, $u['password_hash'])) {
    session_regenerate_id(true);
    $_SESSION['user'] = ['id' => $u['id'], 'name' => $u['name'], 'email' => $u['email'], 'role'
=> $u['role']];
```

```
    $_SESSION['last_activity'] = time();
    $pdo->prepare('UPDATE users SET last_login = NOW() WHERE id = ?')->execute([$u['id']]);

    if (!empty($_POST['remember'])) {
        $sel = bin2hex(random_bytes(12));
        $tok = bin2hex(random_bytes(32));
        $pdo->prepare('INSERT INTO remember_tokens (user_id, selector, token_hash, expires_at)
VALUES (?, ?, ?, DATE_ADD(NOW(), INTERVAL 14 DAY))')
            ->execute([$u['id'], $sel, hash('sha256', $tok)]);
        setcookie('remember', "$sel:$tok", ['expires' => time() + 14*86400, 'path' => '/',
'httponly' => true, 'samesite' => 'Lax']);
    }
    redirect($u['role'] === 'admin' ? 'admin/dashboard.php' : 'dashboard.php');
}
```

3. Verification Screenshot: Login Portal & Demo Helper

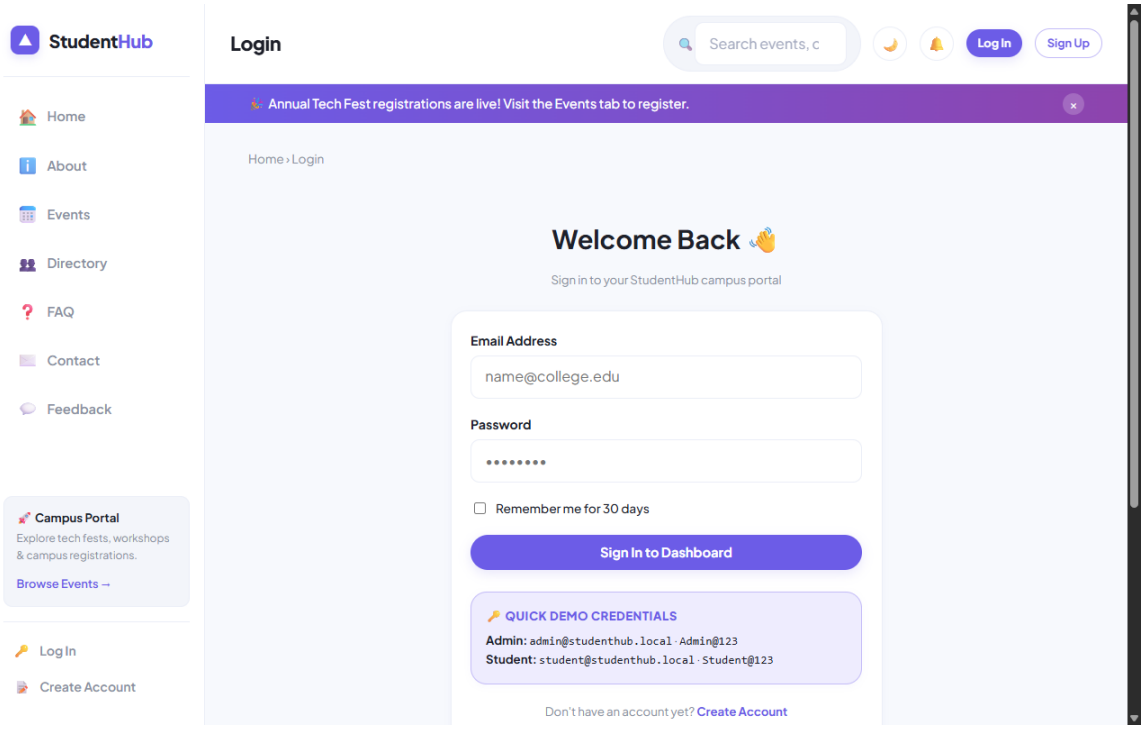


Figure: Secure Login Screen with Role Credentials Helper and Remember-Me Toggle

4. Verification Screenshot: Student Dashboard (Dhruv Bhoi)

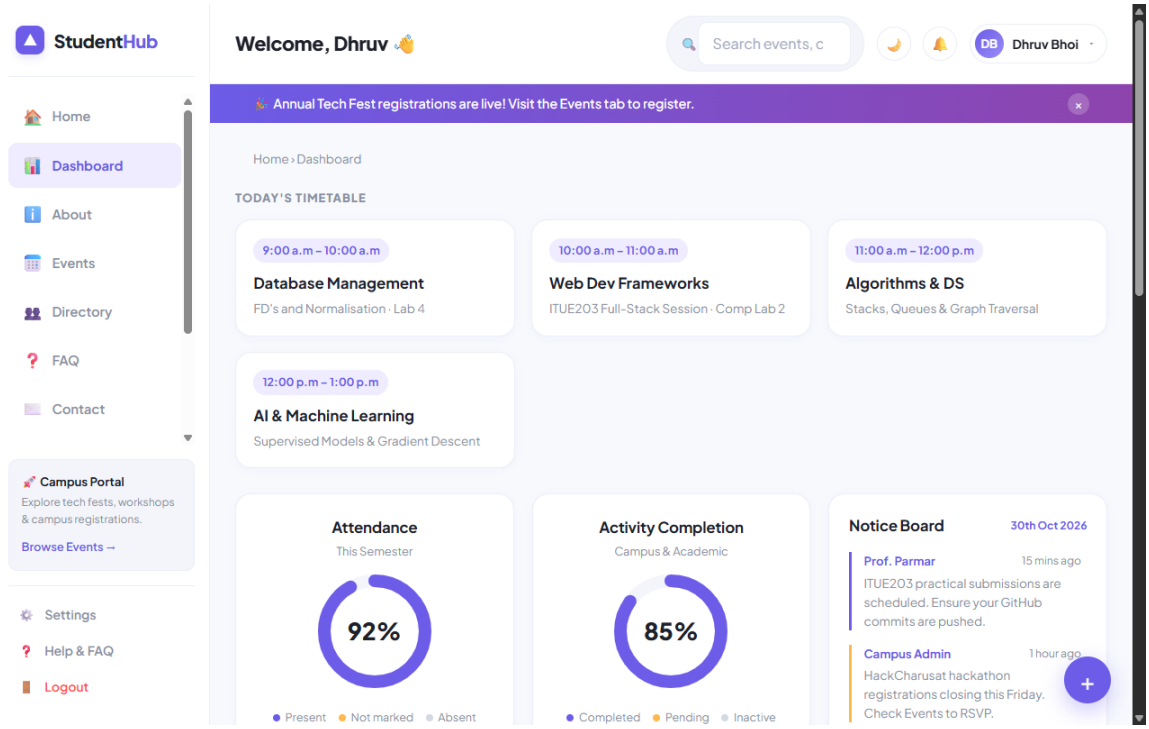


Figure: Authenticated Student Dashboard with Timetable, Progress Donuts, and Events Feed

5. Key Questions & Analysis

Q: How does `session_regenerate_id(true)` protect against session fixation attacks?

Ans: In a session fixation attack, an adversary tricks the victim into using a predetermined session ID. Calling `session_regenerate_id(true)` issues a brand new cryptographically random session identifier while deleting the old session storage, invalidating any fixation token held by the attacker.

Q: How does the split-token 'Remember Me' mechanism prevent database leakage attacks?

Ans: The database stores only the public selector and a SHA-256 hash of the validator token. If an attacker breaches the database, they cannot forge session cookies because the raw validator token is never stored server-side.

Practical 10 Conclusion & Learning Outcomes

Implemented secure user authentication utilizing PHP sessions, session fixation countermeasures (session_regenerate_id), 15-minute inactivity timeouts, role-based access control (RBAC), and split-token remember-me persistent cookies with SHA-256 database hashing.